JAVA

INTERVIEW PREPARATION GUIDE

Core Java Levels I, II & III

140+ Questions with Detailed Answers, Examples & Cheat Sheet

★ **Frequently Asked Questions are highlighted**

Table of Contents

Table of Contents....2
Table of Contents....2
Step 1 – Core Java — Level I (Basic)....3
Step 1 – Core Java — Level I (Basic)....3
Step 2 – Core Java — Level II (Intermediate)....13
Step 2 – Core Java — Level II (Intermediate)....13
Core Concepts....13
Core Concepts....13
OOP / Keywords....16
OOP / Keywords....16
Static / Final....17
Static / Final....17
Functional Programming — Java 8....20
Functional Programming — Java 8....20
Collections....21
Collections....21
Streams / Java Versions....23
Streams / Java Versions....23
Misc / Advanced....25
Misc / Advanced....25
Serialization....28
Serialization....28
Overloading / Overriding....29
Overloading / Overriding....29
Step 3 – Core Java — Level III (Advanced)....32
Step 3 – Core Java — Level III (Advanced)....32
Core / Design....32
Core / Design....32
Advanced OOP / Packages....34
Advanced OOP / Packages....34
Concurrency....36
Concurrency....36
Executor Framework....38
Executor Framework....38
Collections / Concurrency....38
Collections / Concurrency....38
Java 8 Streams....39
Java 8 Streams....39
Serialization / Generics....41
Serialization / Generics....41
★ QUICK REFERENCE CHEAT SHEET ★....44
★ QUICK REFERENCE CHEAT SHEET ★....44

Step 1 – Core Java — Level I (Basic)

★ FAQ Q1. What is Java?

- Java is a high-level, class-based, object-oriented programming language developed by James Gosling at Sun Microsystems in 1995.
- Follows Write Once, Run Anywhere (WORA) — compiled to bytecode executed on JVM.
- Platform-independent, strongly typed, garbage-collected, multithreaded.
- Current steward: Oracle Corporation. Latest LTS: Java 21.

SUMMARY *Java = OOP + platform-independent + bytecode + JVM + WORA. Strongly typed, garbage-collected language.*

★ FAQ Q2. What are the features of Java?

- Simple — familiar C/C++ syntax, no pointers, automatic memory management.
- Object-Oriented — encapsulation, inheritance, polymorphism, abstraction.
- Platform Independent — bytecode runs on any JVM.
- Secure — no explicit pointer arithmetic; bytecode verified before execution.
- Robust — strong exception handling, type checking, garbage collection.
- Multithreaded — built-in Thread support and concurrency utilities.
- High Performance — JIT compiler converts bytecode to native code at runtime.
- Distributed — supports RMI and socket programming.
- Dynamic — classes loaded on demand via ClassLoader.

SUMMARY *Key features: Simple, OOP, Platform-independent, Secure, Robust, Multithreaded, High Performance.*

★ FAQ Q3. What is JVM and why is it important?

- JVM (Java Virtual Machine) is the runtime engine that executes Java bytecode.
- It converts .class bytecode to machine-specific native instructions using the JIT compiler.
- Provides memory management (heap, stack, method area), garbage collection, and security.
- Platform independence: different JVM implementations exist for each OS, but all execute the same bytecode.
- Key subsystems: ClassLoader, Bytecode Verifier, Execution Engine (JIT + Interpreter), Garbage Collector.

SUMMAR *JVM is the engine that runs Java bytecode, providing platform independence, memory management, and security.*

Y	

★ FAQ Q4. What is the difference between JDK, JRE, and JVM?

Component	Description
JDK	Java Development Kit — full toolkit for developing Java apps (includes JRE + compiler javac, javadoc, jar, debugger).
JRE	Java Runtime Environment — provides libraries and JVM to run compiled Java programs. No development tools.
JVM	Java Virtual Machine — the runtime that interprets/JIT-compiles bytecode. Part of JRE.

Relationship: JDK ⊃JRE ⊃JVM

SUMMARY	*JDK = develop+run; JRE = run only; JVM = execute bytecode. JDK includes JRE which includes JVM.*

★ FAQ Q5. What is the use of public static void main(String[] args)?

- public — JVM must call it from outside the class; must be accessible.
- static — JVM calls it without creating an instance of the class.
- void — returns nothing to JVM.
- main — convention; JVM looks for this exact name.
- String[] args — command-line arguments passed to the program.

```
public static void main(String[] args) { System.out.println("Hello, Java!"); }
```

SUMMARY	*Entry point of every Java app. static so JVM can call without object; public for JVM access; void as JVM expects no return.*

★ FAQ Q6. Explain the concept of Object-Oriented Programming in Java.

- OOP organizes software around objects (data + behavior) rather than functions.
- Four pillars:
  - 1. Encapsulation — bundling data and methods, hiding internal state (private fields + getters/setters).
  - 2. Inheritance — subclass acquires properties/methods of superclass using extends.
  - 3. Polymorphism — same interface for different types (overloading = compile-time; overriding = runtime).
  - 4. Abstraction — hiding implementation details, exposing only what is necessary (abstract class / interface).
- Benefits: Reusability, Modularity, Extensibility, Maintainability.

SUMMARY	*OOP in Java is built on 4 pillars: Encapsulation, Inheritance, Polymorphism, Abstraction for modular & reusable code.*

★ FAQ Q7. What is inheritance in Java?

- Mechanism where a subclass acquires properties and methods of a superclass.
- Keyword: extends (class), implements (interface).
- Java supports single inheritance for classes (no multiple class inheritance).
- Supports multilevel and hierarchical inheritance.
- Method overriding enables runtime polymorphism in inherited hierarchies.

```
class Animal { void speak() { System.out.println("..."); } }
class Dog extends Animal { @Override void speak() { System.out.println("Woof"); } }
```

SUMMARY

Inheritance = subclass extends superclass to reuse code. Java allows single class inheritance; multiple via interfaces.

★ FAQ Q8. What is polymorphism in Java?

- Polymorphism = "many forms" — the same method/interface behaves differently based on the object.
- Compile-time (static) polymorphism: Method Overloading — same method name, different parameters.
- Runtime (dynamic) polymorphism: Method Overriding — subclass overrides superclass method; resolved via virtual dispatch.

```
Animal a = new Dog(); a.speak(); // calls Dog.speak() at runtime
```

SUMMARY

Polymorphism: compile-time via overloading; runtime via overriding. Enables flexible and extensible designs.

Q9. Explain encapsulation with an example in Java.

- Encapsulation = binding data (fields) and methods together, controlling access via access modifiers.
- Fields are made private; access controlled through public getter/setter methods.
- Benefits: Data hiding, Validation in setters, Easier to change implementation.

```
public class Person {
  private String name;
  public String getName() { return name; }
  public void setName(String n) { this.name = n; }
}
```

SUMMARY

Encapsulation: private fields + public getters/setters. Provides data hiding and controlled access to state.

★ FAQ Q10. What is an interface in Java?

- An interface is a contract — defines method signatures without implementation (prior to Java 8).

- Since Java 8: can have default and static methods with implementation.
- Since Java 9: can have private methods.
- A class implements an interface using the implements keyword.
- A class can implement multiple interfaces (solves Java's no-multiple-inheritance limitation).
- All fields in an interface are public static final (constants) by default.

```
interface Drawable { void draw(); default void print()
{ System.out.println("Drawing"); } }
```

SUMMARY	*Interface = pure contract (abstract methods + constants). Java 8+ adds default/static methods. A class can implement many interfaces.*

★ FAQ Q11. Explain the concept of an abstract class.

- An abstract class cannot be instantiated; it may contain both abstract and concrete methods.
- Declared with the abstract keyword.
- Subclasses must implement all abstract methods or be declared abstract themselves.
- Can have constructors, instance variables, and non-abstract methods.
- Use when: partial implementation is shared among related classes.

Feature	Abstract Class vs Interface
Instantiation	Cannot instantiate either
Methods	Can have concrete methods; Interface: default only in Java 8+
Variables	Can have instance vars; Interface: only static final
Inheritance	Single (extends); Interface: multiple (implements)
SUMMARY	*Abstract class: partial implementation blueprint. Use when classes share behavior. Interface: pure contract for unrelated classes.*

Q12. What are constructors in Java?

- A constructor is a special method used to initialize an object when it is created.
- Same name as the class; no return type (not even void).
- Types: Default constructor (no-arg), Parameterized constructor.
- If no constructor is defined, Java provides a default no-arg constructor.
- Constructor chaining: this() calls same-class constructor; super() calls parent constructor.

```
class Car { String model; Car(String m) { this.model = m; } }
```

SUMMARY	*Constructors initialize objects. Same name as class, no return type. Java provides default if none defined.*

★ FAQ Q13. What is method overloading?

- Method overloading = multiple methods with the same name but different parameter lists in the same class.
- Resolved at compile time (static/early binding).

- Can differ in: number of parameters, parameter types, parameter order.
- Return type alone is NOT sufficient to overload.

```
int add(int a, int b) { return a+b; }
double add(double a, double b) { return a+b; }
```

SUMMARY

Overloading: same method name, different parameters in same class. Compile-time polymorphism.

★ FAQ Q14. What is method overriding?

- Method overriding = subclass provides a specific implementation of a method already defined in its superclass.
- Resolved at runtime (dynamic/late binding) based on actual object type.
- Rules: same name, same parameter list, same or covariant return type; not stricter access modifier; cannot override final/static/private methods.
- @Override annotation is recommended to detect errors at compile time.

```
class Animal { void sound() { System.out.println("Generic"); } }
class Cat extends Animal { @Override void sound() { System.out.println("Meow"); } }
```

SUMMARY

Overriding: subclass redefines superclass method with same signature. Runtime polymorphism; use @Override annotation.

Q15. What is a package in Java?

- A package is a namespace that organizes related classes and interfaces.
- Provides access control, avoids naming conflicts.
- Built-in packages: java.lang, java.util, java.io, java.net, etc.
- Declare with package keyword at top of file; import with import keyword.

```
package com.example.app;
import java.util.List;
```

SUMMARY

Package = namespace for organizing classes. Avoids naming conflicts; controls access. Declared at top of file.

★ FAQ Q16. Explain the final keyword in Java.

- final variable — value cannot change (effectively a constant).
- final method — cannot be overridden by subclasses.
- final class — cannot be subclassed (e.g., String, Integer).
- final parameters — parameter value cannot be reassigned inside the method.
- Used for immutability, security, and performance optimization.

SUMMARY	*final: prevents change. Variable = constant; Method = no override; Class = no inheritance. Promotes immutability.*

★ FAQ Q17. What are Java Exceptions?

- An exception is an event that disrupts normal program flow during execution.
- Hierarchy: Throwable → Error | Exception → RuntimeException | Checked exceptions.
- Handled using: try, catch, finally, throw, throws.
- finally block always executes (used for cleanup like closing resources).
- try-with-resources (Java 7+) auto-closes Closeable resources.

SUMMARY	*Exceptions disrupt program flow. Handled via try-catch-finally. Use checked for recoverable; unchecked for programmer errors.*

★ FAQ Q18. What is the difference between checked and unchecked exceptions?

Aspect	Checked vs Unchecked
Checked	Must be caught or declared with throws. Checked at compile time (e.g., IOException, SQLException).
Unchecked	Extend RuntimeException. Not required to be caught. E.g., NullPointerException, ArrayIndexOutOfBoundsException.
Error	Extend Error. JVM-level issues (e.g., OutOfMemoryError, StackOverflowError). Should not be caught.
Best Practice	Use checked for recoverable situations; unchecked for programming errors.
SUMMARY	*Checked: compile-time, must handle (IOException). Unchecked: runtime, programmer bugs (NullPointerException). Error: JVM-level.*

★ FAQ Q19. What is the static keyword used for in Java?

- static variable — shared across all instances of a class (class variable).
- static method — belongs to the class, not instances; can be called without creating an object.
- static block — runs once when the class is loaded; used for static initialization.
- static nested class — nested class that does not need an outer class instance.
- static methods cannot access non-static (instance) members directly.

SUMMARY	*static = belongs to the class, not instances. Shared state (variable), utility methods, class-level initialization (block).*

★ FAQ Q20. What is a thread in Java?

- A thread is the smallest unit of execution within a process.
- Java supports multithreading: multiple threads run concurrently within the same JVM process.
- Created by: extending Thread class OR implementing Runnable interface.
- Thread lifecycle: NEW → RUNNABLE → BLOCKED/WAITING/TIMED_WAITING → TERMINATED.

```
new Thread(() -> System.out.println("Thread running")).start();
```

SUMMARY	*Thread = lightweight execution unit. Created via Thread or Runnable. Lifecycle: New->Runnable->Running->Terminated.*

★ FAQ Q21. Explain the difference between == and .equals() in Java.

Operator	Behavior
==	Compares references (memory addresses) for objects; compares values for primitives.
.equals()	Compares content/value of objects. Overridden in String, Integer, etc.
Example	String a = new String("hi"); String b = new String("hi"); a==b → false; a.equals(b) → true
Note	Always use .equals() to compare object content. Use == only for reference/null checks.
SUMMARY	*== checks reference equality; .equals() checks value/content equality. Always use .equals() to compare object values.*

★ FAQ Q22. What is garbage collection in Java?

- Garbage Collection (GC) automatically reclaims memory used by objects that are no longer reachable.
- Programmer does not manually free memory (unlike C/C++).
- JVM tracks object references; collects objects with no live references.
- GC algorithms: Serial, Parallel, G1 (default in Java 9+), ZGC, Shenandoah.
- Heap divided into: Young Generation (Eden + Survivor spaces) and Old Generation.
- System.gc() suggests GC but does not guarantee it runs.

SUMMARY	*GC automatically frees unreachable objects from heap memory. Java 9+ uses G1GC by default. Removes need for manual memory management.*

★ FAQ Q23. What is the Collections Framework in Java?

- A unified architecture for storing and manipulating groups of objects.
- Core interfaces: Collection → List, Set, Queue; Map (separate hierarchy).
- Common implementations: ArrayList, LinkedList, HashSet, TreeSet, HashMap, TreeMap, PriorityQueue.
- Utility class: Collections (sort, shuffle, binarySearch, etc.).
- Java 8+: Stream API integration for functional-style operations.

SUM	*Collections Framework: unified API for data structures. Key interfaces: List, Set, Queue, Map. Use ArrayList, HashMap most commonly.*

MARY

★ FAQ Q24. Explain synchronized keyword in Java.

- synchronized ensures that only one thread at a time can execute a block/method for a given object monitor.
- Synchronized method: locks the current object (this) or the class for static methods.
- Synchronized block: more granular; locks specified object.
- Prevents race conditions and ensures memory visibility.
- Drawback: performance overhead due to lock acquisition.

```
synchronized void increment() { count++; }
synchronized(this) { sharedResource.update(); }
```

SUMMARY

synchronized prevents concurrent access to shared resources. Locks object/class monitor. Use for thread-safe operations.

★ FAQ Q25. What are generics in Java?

- Generics allow you to write type-safe code with parameterized types.
- Eliminated ClassCastException at runtime by catching type errors at compile time.
- Used in collections: List<String>, Map<String, Integer>.
- Wildcard: ? (unknown type), ? extends T (upper bound), ? super T (lower bound).
- Type erasure: generic type info is removed at runtime; replaced with Object or bound type.

```
public <T> T getFirst(List<T> list) { return list.get(0); }
```

SUMMARY

Generics provide type safety at compile time. Enable reusable type-parameterized classes/methods. Type info erased at runtime.

Q26. What is the use of this keyword in Java?

- this refers to the current object instance.
- Disambiguates instance variable from parameter with same name.
- this() — calls another constructor in the same class (constructor chaining).
- Can be passed as argument or returned from method.

```
class Point { int x; Point(int x) { this.x = x; } }
```

SUMMARY

this = reference to current object. Used to disambiguate fields, chain constructors, and pass current instance.

Q27. What is Enum in Java?

- An enum (enumeration) is a special class representing a group of named constants.
- Type-safe alternative to int constants.
- Can have fields, methods, and constructors.
- Implicitly extends java.lang.Enum; cannot extend another class.
- Useful in switch statements, strategy patterns.

```
enum Day { MON, TUE, WED, THU, FRI, SAT, SUN }
Day d = Day.MON; if (d == Day.MON) { ... }
```

SUMMARY *Enum = type-safe named constants. Can have fields/methods. Implicitly extends Enum class. Thread-safe singletons.*

Q28. What are threads?

- A thread is a lightweight subprocess — the smallest unit of CPU scheduling.
- Shares the same process memory space (heap) but has its own stack and program counter.
- Enables concurrent execution within a program.

SUMMARY *Thread = smallest execution unit sharing process memory. Enables concurrency. Each has own stack and registers.*

★ FAQ Q29. What is multithreading?

- Multithreading = executing multiple threads concurrently within a single process.
- Benefits: Better CPU utilization, responsive applications, parallel processing.
- Challenges: Race conditions, deadlocks, data inconsistency.
- Java provides: Thread class, Runnable, ExecutorService, ForkJoinPool, CompletableFuture.

SUMMARY *Multithreading = concurrent execution of multiple threads. Maximizes CPU use but requires synchronization to avoid race conditions.*

★ FAQ Q30. Explain volatile keyword in Java.

- volatile ensures that a variable's value is always read from and written to main memory (not CPU cache).
- Prevents thread caching issues; guarantees visibility across threads.
- Does NOT make compound operations (e.g., i++) atomic.
- Lighter than synchronized; use when only visibility (not atomicity) is needed.
- Common use: flag variables shared between threads.

```
volatile boolean running = true; // visible to all threads
```

SU *volatile guarantees memory visibility across threads. Not atomic. Use for simple flags; use synchronized or AtomicInteger for compound ops.*

M
M
A
R
Y

Step 2 – Core Java — Level II (Intermediate)

Core Concepts

★ FAQ Q1. What happens if we don't declare main as static?

- JVM cannot call main() without creating an object of the class.
- Result: java.lang.NoSuchMethodError: main — JVM cannot find the static entry point.
- The class would compile fine but fail at runtime with an error.

SUMMARY

Without static, JVM cannot call main without an instance. Compiles but fails at runtime with NoSuchMethodError.

Q2. Can we override the main method?

- No. main() is static, and static methods cannot be overridden (only hidden).
- Defining main() in a subclass hides the parent's main — it does NOT override.
- JVM always invokes the class whose main() is specified on the command line.

SUMMARY

Static methods cannot be overridden — they can only be hidden. JVM always calls main() of the class specified.

Q3. Can we overload the main method?

- Yes, Java allows overloading — you can define multiple main() methods with different parameters.
- JVM will only call the specific main(String[] args) signature as the entry point.
- Other overloaded versions must be called explicitly from within code.

SUMMARY

Yes, main() can be overloaded, but JVM only calls main(String[] args) as the program entry point.

Q4. Can JVM execute overloaded main method?

- No. JVM specifically looks for public static void main(String[] args).
- Overloaded versions are treated as regular methods, not program entry points.

SUMMA

JVM only calls main(String[] args) as entry point. Overloaded versions are ordinary methods — not entry points.

R Y	

★ FAQ Q5. Difference between primitive and non-primitive data types?

Aspect	Primitive vs Non-Primitive
Primitives	byte, short, int, long, float, double, char, boolean. Stored on stack; hold actual values.
Non-Primitives	Objects, arrays, Strings. Stored on heap; variable holds a reference.
Null	Primitives cannot be null; non-primitives can.
Default	Primitives have defaults (int=0, boolean=false); objects default to null.
SUMMARY	*Primitives: 8 types, value-based, stack-stored, cannot be null. Non-primitives: objects on heap, reference-based, nullable.*

Q6. Can primitive data types be null?

- No. Primitive types cannot hold null; they always have a default value.
- Null can only be assigned to reference types (objects).
- Use wrapper classes (Integer, Boolean) when null is needed for primitives.

S U M M A R Y	*Primitives cannot be null — they have default values (0, false, etc.). Use wrapper classes if null representation is needed.*

Q7. Can we declare pointers in Java?

- No. Java does not support explicit pointers for security and safety reasons.
- Java uses references, which are safer, managed pointers handled by the JVM.
- Prevents pointer arithmetic and direct memory access bugs.

S U M M A R Y	*Java has no explicit pointers. References are managed by JVM — safer alternative without arithmetic or direct memory access.*

★ FAQ Q8. Difference between == and .equals()?

- == compares object references (memory addresses); returns true only if both variables point to the same object.
- .equals() compares logical/content equality; overridden in String, Integer, etc.
- For primitives, == compares values directly.
- Best practice: always use .equals() for object content comparison; never == for Strings.

S U M M A R	*== is reference equality; .equals() is content equality. Override .equals() in custom classes for correct behavior.*

Y

★ FAQ Q9. What are wrapper classes?

- Wrapper classes convert primitive types into objects: Integer, Long, Double, Boolean, Character, Byte, Short, Float.
- Located in java.lang package.
- Enable primitives to be used in Collections (which require objects).
- Provide utility methods: Integer.parseInt(), Double.valueOf(), etc.

SUMMARY

Wrapper classes = OO representations of primitives. Enable primitives in collections; provide utility parsing/conversion methods.

Q10. Why do we need wrapper classes?

- Collections (ArrayList, HashMap) work only with objects, not primitives.
- Provide useful static utility methods (parsing, conversion, constants like Integer.MAX_VALUE).
- Support null values when dealing with database results or optional data.
- Enable reflection and generics which work with Object types.

SUMMARY

Wrapper classes needed for: Collections (require objects), utility methods, nullable primitives, generics support.

Q11. Why wrapper classes are used in collections?

- Java collections are generic and work with Objects; primitive types are not objects.
- Autoboxing converts primitives to wrappers automatically when added to collections.

```
List<Integer> list = new ArrayList<>(); list.add(42); // autoboxed to Integer
```

SUMMARY

Collections require objects, not primitives. Wrapper classes bridge this gap. Autoboxing handles conversion automatically.

★ FAQ Q12. Difference between autoboxing and unboxing?

- Autoboxing: automatic conversion of primitive to wrapper object by the compiler.
- Unboxing: automatic conversion of wrapper object back to primitive.

```
Integer i = 5;     // autoboxing: int 5 → Integer
int x = i;         // unboxing: Integer → int
```

- Beware: unboxing null wrapper throws NullPointerException.

SUM

Autoboxing: primitive → wrapper automatically. Unboxing: wrapper → primitive. NullPointerException risk when unboxing null.

MARY

OOP / Keywords

★ FAQ Q13. How does @Override annotation influence method overriding?

- @Override is a compile-time annotation that tells the compiler this method is intended to override a superclass method.
- If the method does NOT actually override (typo, wrong signature), compiler throws an error.
- No runtime impact — purely a safety net for developers.

SUMMARY *@Override causes compile-time verification that the method truly overrides a parent method. Prevents subtle bugs from signature mismatches.*

Q14. What happens if a superclass method is overridden by multiple subclasses?

- Each subclass provides its own implementation; they are independent.
- At runtime, the JVM calls the implementation of the actual object type (dynamic dispatch).
- This is the basis of runtime polymorphism — same reference, different behavior.

SUMMARY *Each subclass has its own independent override. JVM picks the correct implementation at runtime based on actual object type.*

Q15. What are this and super keywords?

- this: refers to the current class instance. Used for disambiguation, constructor chaining.
- super: refers to the immediate parent class. Used to call parent methods/constructors.
- super() must be the first statement in a constructor if used.
- Neither can be used in static methods (they require an instance context).

SUMMARY *this = current object; super = parent class. super() calls parent constructor (must be first). Neither works in static context.*

Q16. Can this be assigned a new value?

- No. this is a final implicit reference; you cannot reassign it.
- Attempting this = someObject; results in a compile-time error.

SUM *this is an immutable reference — cannot be reassigned. It always refers to the current object.*

MARY	

Q17. What happens if super is used without superclass?

- Every class implicitly extends Object, so super always refers to at least Object.
- Calling super.method() where no superclass defines it will call Object's version.

SUMMARY	*super always refers to at least Object (Java's root class). super.method() calls the nearest parent's version.*

Q18. Can this or super be used in static methods?

- No. this and super require an object instance (implicit parameter).
- Static methods belong to the class, not instances. Using them causes a compile error.

SUMMARY	*this and super cannot be used in static methods — no instance context exists in static scope.*

Q19. How does super help in polymorphism?

- super.method() explicitly calls the parent class version, bypassing the overridden version in the subclass.
- Allows subclass to extend parent behavior: call super.method() then add own logic.
- Essential in overriding chains where parent behavior must be preserved.

SUMMARY	*super.method() calls parent's implementation from an overriding subclass. Enables extending (not replacing) parent behavior.*

Static / Final

★ FAQ Q20. What is static keyword?

- static members belong to the class, not to any particular object instance.
- static variable: single copy shared by all instances (class variable).
- static method: callable on the class itself; cannot access instance members.
- static block: runs once at class loading for initialization.
- static nested class: doesn't need outer class instance.

SUMM	*static = class-level, not instance-level. Shared variables, utility methods, class-level initialization via static blocks.*

A
R
Y

Q21. Can a static block throw exception?

- Static blocks can throw unchecked exceptions freely.
- For checked exceptions, they must be caught inside the static block itself.
- If a static block throws an uncaught exception, it wraps in ExceptionInInitializerError.

S
U
M
M
A
R
Y

Static blocks can throw unchecked exceptions. Checked exceptions must be caught inside. Uncaught exceptions → ExceptionInInitializerError.

★ FAQ Q22. Can we override static methods?

- No. Static methods cannot be overridden — they are resolved at compile time based on reference type (not object type).
- A subclass can define a static method with the same signature — this is method hiding, not overriding.
- No dynamic dispatch for static methods.

S
U
M
M
A
R
Y

Static methods cannot be overridden (no polymorphism). Redefining in subclass = method hiding. Always called based on reference type.

Q23. Can static method access non-static members?

- No. Static methods do not have access to this; they cannot reference instance variables or instance methods directly.
- Can access them via an object reference created within the static method.

```
static void show() { MyClass obj = new MyClass(); obj.instanceMethod(); } // OK
```

S
U
M
M
A
R
Y

Static methods cannot directly access instance members (no this). Can access them via an explicit object reference.

Q24. What is static block?

- A static block (static initializer) is executed once when the class is first loaded by ClassLoader.
- Used to initialize static variables or perform one-time setup.
- Runs before any constructor or main method.

```
static { System.out.println("Class loaded"); counter = 10; }
```

S
U
M
M

Static block runs once on class loading before constructors. Used for complex static variable initialization.

A
R
Y

Q25. Can we print without main method?

- In Java 7 and earlier, using a static block: possible as JVM used to run static blocks before checking for main.
- From Java 7+: JVM checks for main after class loading; prints then throws NoSuchMethodError.
- In Java 8+: no, program requires main. Static blocks run but JVM exits with error.

S
U
M
M
A
R
Y

Pre-Java 7: yes via static block. Java 7+: static block runs but JVM throws NoSuchMethodError after. Not practically possible.

★ FAQ Q26. What is final keyword?

- final variable: cannot be reassigned after initialization.
- final method: cannot be overridden in a subclass.
- final class: cannot be extended (e.g., String, Math, Integer).

S
U
M
M
A
R
Y

final: variable=constant, method=no override, class=no inheritance. Core tool for immutability and security.

Q27. Use cases of final keyword?

- Defining constants: public static final double PI = 3.14159;
- Creating immutable classes (all fields final, no setters).
- Security: prevent method behavior from being altered by subclasses.
- Performance: JVM can inline final methods.

S
U
M
M
A
R
Y

final used for constants, immutable classes, preventing inheritance/overriding, and JVM performance optimization.

★ FAQ Q28. How final helps in immutability & thread safety?

- final fields are guaranteed to be fully initialized before any thread can see the object.
- Immutable objects can be safely shared between threads without synchronization.
- String is final and immutable — safe for use as HashMap keys, in thread pools, etc.
- Combined with private, no-setters, and defensive copying for fully immutable class.

S
U
M

final fields = safe publishing in multithreaded code. Immutable objects need no synchronization — inherently thread-safe.

MARY

Functional Programming — Java 8

★ FAQ Q29. What is functional interface?

- A functional interface has exactly one abstract method.
- Can be annotated with @FunctionalInterface (enforced by compiler).
- Enables lambda expressions and method references.
- Built-in: Runnable, Callable, Comparator, Function<T,R>, Predicate<T>, Consumer<T>, Supplier<T>.

```
@FunctionalInterface interface Greeter { void greet(String name); }
Greeter g = name -> System.out.println("Hello " + name);
```

SUMMARY

Functional interface = exactly 1 abstract method. Foundation for lambdas. Key built-ins: Function, Predicate, Consumer, Supplier.

Q30. Can functional interface extend another interface?

- Yes, but the extending interface must still have exactly one abstract method total.
- It can inherit abstract method from parent and add default/static methods.
- If it adds another abstract method, it is no longer a functional interface.

SUMMARY

Functional interface can extend another interface as long as exactly ONE abstract method remains in total.

Q31. Advantages of functional interface?

- Enables lambdas and method references — concise, readable code.
- Supports functional programming style in Java.
- Reduces boilerplate (no need for anonymous inner classes).
- Enables composition: Function.andThen(), Predicate.and(), etc.

SUMMARY

Functional interfaces enable lambda expressions, method references, and functional composition — reducing boilerplate significantly.

★ FAQ Q32. What are Java 8 features?

- Lambda Expressions — concise anonymous function syntax.
- Stream API — functional-style processing of collections.

- Optional<T> — wrapper to avoid NullPointerException.
- Default and Static methods in interfaces.
- Method References (::) — shorthand for lambdas calling existing methods.
- New Date/Time API (java.time) — replaces legacy java.util.Date.
- CompletableFuture — async/non-blocking programming.
- Collectors, Predicate, Function, Consumer, Supplier improvements.

SUMMARY	*Java 8: Lambdas, Streams, Optional, default interface methods, method references, new Date/Time API, CompletableFuture.*

Q33. Why Optional, Lambda, Streams introduced?

- Lambda: reduce verbosity of anonymous inner classes; enable functional programming.
- Streams: declarative, readable collection processing with lazy evaluation and parallel support.
- Optional: explicit, type-safe handling of nullable values to reduce NullPointerExceptions.

SUMMARY	*Lambdas reduce verbosity; Streams enable declarative data processing; Optional makes null handling explicit and safe.*

Collections

★ FAQ Q34. What is Collection Framework?

- A unified architecture providing interfaces, implementations, and algorithms to store and manipulate object groups.
- Core hierarchy: Iterable → Collection → List / Set / Queue; Map is separate.
- Benefits: Reduces programming effort, increases performance, provides interoperability.

SUMMARY	*Collections Framework = standard data structure library. Provides interfaces (List, Set, Map, Queue) and implementations.*

Q35. Main interfaces of Collection Framework?

Interface	Description
List	Ordered, allows duplicates. ArrayList, LinkedList, Vector.
Set	Unordered (or sorted), no duplicates. HashSet, TreeSet, LinkedHashSet.
Queue	FIFO ordering. LinkedList, PriorityQueue, ArrayDeque.
Deque	Double-ended queue. ArrayDeque.
Map	Key-value pairs, no duplicate keys. HashMap, TreeMap, LinkedHashMap, Hashtable.
SUMMARY	*Key interfaces: List (ordered+duplicates), Set (no duplicates), Queue (FIFO), Map (key-value). Each has*

multiple implementations.

Q36. How Iterator works?

- Iterator provides a standard way to traverse elements of a collection without exposing its underlying structure.
- Methods: hasNext() — returns true if more elements; next() — returns next element; remove() — removes last returned element.
- Fail-fast: throws ConcurrentModificationException if collection is modified during iteration (structural change).

```
Iterator<String> it = list.iterator(); while (it.hasNext()) { String s = it.next(); }
```

SUMMARY

Iterator = cursor for traversing collections. hasNext()/next()/remove(). Fail-fast — throws CME if modified during iteration.

Q37. Common methods in collections?

- Collection: add(), remove(), contains(), size(), isEmpty(), clear(), iterator().
- List: get(index), set(index, e), indexOf(), subList().
- Map: put(), get(), containsKey(), containsValue(), keySet(), values(), entrySet().
- Collections utility: sort(), reverse(), shuffle(), min(), max(), frequency().

SUMMARY

Core methods: add/remove/contains for Collection; get/set/indexOf for List; put/get/keySet for Map.

Q38. How Collection Framework handles concurrency?

- Legacy thread-safe: Vector, Hashtable (synchronized, poor performance).
- Synchronized wrappers: Collections.synchronizedList(), synchronizedMap() — coarse locking.
- Concurrent collections (java.util.concurrent): ConcurrentHashMap (segment/stripe locking), CopyOnWriteArrayList, ConcurrentLinkedQueue, BlockingQueue.
- ConcurrentHashMap: allows concurrent reads and fine-grained locking for writes.

SUMMARY

For thread-safety use java.util.concurrent collections (ConcurrentHashMap, CopyOnWriteArrayList). Avoid Vector/Hashtable (legacy, slow).

Q39. How to choose correct collection?

- Need fast random access → ArrayList.
- Frequent insertions/deletions in middle → LinkedList.
- No duplicates, fast lookup → HashSet.

- No duplicates, sorted order → TreeSet.
- Key-value, fast access → HashMap.
- Key-value, sorted by key → TreeMap.
- FIFO queue → ArrayDeque or LinkedList.
- Priority-based → PriorityQueue.
- Thread-safe map → ConcurrentHashMap.

SUMMARY	*Choose by use case: ArrayList for access, LinkedList for insertions, HashSet for uniqueness, HashMap for key-value, ConcurrentHashMap for threads.*

Q40. Enhancements in Java 8 Collections?

- Map.getOrDefault(), putIfAbsent(), computeIfAbsent(), merge(), replaceAll().
- List.sort(), removeIf(), forEach().
- Iterable.forEach() — iterate with lambda.
- Stream support on all collections via stream() and parallelStream().

SUMMARY	*Java 8 adds forEach, removeIf, stream(), Map compute/merge operations — enabling functional-style collection processing.*

Q41. Difference between Iterator and ListIterator?

Feature	Iterator vs ListIterator
Direction	Iterator: forward only. ListIterator: forward AND backward.
Modification	Iterator: remove only. ListIterator: add, set, remove.
Position	Iterator: no index. ListIterator: provides nextIndex(), previousIndex().
Applicable	Iterator: any Collection. ListIterator: only List.
SUMMARY	*ListIterator extends Iterator with bidirectional traversal, add/set support, and index access. Only available for List.*

Streams / Java Versions

Q42. Features of Java 17?

- LTS (Long-Term Support) release.
- Sealed Classes (JEP 409): restricts which classes can extend/implement a type.
- Pattern Matching for instanceof (JEP 394): eliminates explicit cast.
- Records finalized (JEP 395): concise immutable data classes.
- Text Blocks finalized (JEP 378): multiline string literals.
- Removal of deprecated APIs (Applet, RMI Activation, Security Manager).

SUMM	*Java 17 LTS: sealed classes, pattern matching for instanceof, records, text blocks all finalized.*

ARY	

★ FAQ Q43. Features of Java 21?

- LTS release with major language and runtime improvements.
- Virtual Threads (JEP 444): lightweight threads for high-concurrency (Project Loom). Thousands of threads without OS thread overhead.
- Sequenced Collections (JEP 431): SequencedCollection interface with first/last access.
- Pattern Matching for switch (JEP 441): type patterns in switch expressions.
- Record Patterns (JEP 440): deconstruct record components in patterns.
- String Templates (preview), Unnamed Patterns/Variables (preview).

SUMMARY	*Java 21 LTS: Virtual Threads (game changer for concurrency), Sequenced Collections, pattern matching for switch, record patterns.*

Q44. Streams vs traditional loop?

Aspect	Streams vs Loops
Style	Streams: declarative (what to do). Loops: imperative (how to do).
Readability	Streams often more concise for complex operations.
Parallel	Streams: easy parallelism via parallelStream(). Loops: manual.
Laziness	Streams are lazy — operations run only when terminal operation triggers.
Debugging	Loops easier to debug. Streams can be harder to step through.
SUMMARY	*Streams: declarative, lazy, parallel-ready. Loops: explicit, easier to debug. Use streams for data transformation pipelines.*

Q45. When to use streams vs loops?

- Use streams for: transformation pipelines, filtering, mapping, collecting results, parallel processing.
- Use loops for: simple iteration, early exit logic (break/continue), complex stateful logic, performance-critical inner loops.

SUMMARY	*Streams for functional pipelines and parallel work. Loops for simple, stateful, or performance-critical iteration.*

★ FAQ Q46. Intermediate vs terminal operations?

- Intermediate operations: lazy, return a new Stream, not executed until terminal op triggered. E.g., filter(), map(), sorted(), distinct(), peek().
- Terminal operations: trigger stream processing, consume the stream. E.g., collect(), forEach(), reduce(), count(), findFirst(), anyMatch().

```
list.stream().filter(x -> x > 5).map(x -> x*2).collect(Collectors.toList());
```

- filter and map are intermediate; collect is terminal.

SUMMARY	*Intermediate ops (filter/map) are lazy and return Stream. Terminal ops (collect/forEach) trigger execution and consume the Stream.*

Q47. Difference in interfaces (Java 7 vs 8)?

- Java 7: interfaces could only have public abstract methods and public static final constants.
- Java 8: added default methods (with implementation) to enable backward-compatible API evolution.
- Java 8: added static methods in interfaces.
- Java 9: added private methods for code reuse within the interface.

SUMMARY	*Java 8 interfaces gained default and static methods. Java 9 added private methods. Enables backward-compatible API evolution.*

Q48. Use of String.join()?

- String.join(delimiter, elements) concatenates strings with a specified delimiter.
- Takes a delimiter and varargs/Iterable of CharSequences.

```
String.join(", ", "Java", "Python", "Go"); // "Java, Python, Go"
String.join("-", List.of("2024","01","15")); // "2024-01-15"
```

SUMMARY	*String.join(delimiter, parts) creates a single string from elements separated by delimiter. Cleaner than manual concatenation.*

Misc / Advanced

★ FAQ Q49. What is Java Memory Model?

- The Java Memory Model (JMM) defines how threads interact through memory — visibility and ordering rules.
- Defines happens-before relationship: actions in one thread that must be visible to another.
- Key rules: synchronized blocks, volatile writes/reads, thread start/join, final field initialization establish happens-before.
- Without proper synchronization, threads may see stale cached values.

SUMMARY	*JMM defines memory visibility and ordering for multithreaded programs via happens-before guarantees. Foundation of Java concurrency model.*

★ FAQ Q50. What is transient keyword?

- transient marks a field to be excluded from serialization.
- When an object is serialized (written to stream), transient fields are skipped.
- Upon deserialization, transient fields receive their default values (null, 0, false).
- Used for: passwords, caches, derived fields, non-serializable members.

```
class User implements Serializable { transient String password; }
```

SUMMARY

transient = skip this field during serialization. Use for sensitive data, caches, or non-serializable fields.

Q51. Can we create server without Spring?

- Yes. Java has built-in capabilities for creating HTTP servers.
- com.sun.net.httpserver.HttpServer — simple lightweight HTTP server in JDK.
- Can also use Java Sockets (ServerSocket) for TCP servers.
- Netty, Undertow, Jetty — third-party options without Spring.
- Java 21+ Project Loom virtual threads make bare-metal servers more competitive.

SUMMARY

Yes, use JDK HttpServer, ServerSocket, or frameworks like Netty/Jetty. Spring simplifies development but is not required.

Q52. What is Exchanger class?

- java.util.concurrent.Exchanger allows exactly two threads to exchange objects at a synchronization point.
- Both threads block at exchange() until the other thread arrives; then they swap their objects.
- Use case: pipeline designs where one thread produces and another consumes, and they need to swap data buffers.

```
Exchanger<String> ex = new Exchanger<>();
Thread 1: String result = ex.exchange("dataFromT1");
Thread 2: String result = ex.exchange("dataFromT2");
```

SUMMARY

Exchanger: synchronization point where exactly 2 threads swap objects. Useful for producer-consumer buffer exchange patterns.

★ FAQ Q53. What is reflection in Java?

- Reflection allows inspection and manipulation of classes, methods, fields, and constructors at runtime.
- Used by: frameworks (Spring, Hibernate), testing tools (JUnit), serialization libraries.
- java.lang.reflect package: Class, Method, Field, Constructor.
- Can invoke private methods, access private fields (with setAccessible(true)).
- Drawback: slower than direct calls; breaks encapsulation; security concerns.

```
Class<?> c = Class.forName("com.example.MyClass");
Method m = c.getDeclaredMethod("secret"); m.setAccessible(true); m.invoke(obj);
```

SUMMARY	*Reflection inspects/modifies class internals at runtime. Powerful but slow and encapsulation-breaking. Used by frameworks internally.*

★ FAQ Q54. Weak vs Soft reference?

Type	Description
Strong	Default. Object reachable via strong reference is never GC-ed.
Soft (SoftReference)	GC-ed only when JVM needs memory (OutOfMemoryError imminent). Good for memory-sensitive caches.
Weak (WeakReference)	GC-ed in next GC cycle regardless of memory. Used in WeakHashMap, listener registries to prevent memory leaks.
Phantom (PhantomReference)	GC-ed; used for pre-finalization cleanup. Reference cleared before finalization.
SUMMARY	*SoftReference: kept until OOM (cache use). WeakReference: cleared on next GC (WeakHashMap). Phantom: post-GC cleanup hook.*

Q55. What is Java Flight Recorder?

- JFR is a low-overhead profiling and diagnostics tool built into the JVM (production-safe).
- Records events: GC, thread activity, I/O, locks, method sampling, CPU usage.
- Integrated with JDK Mission Control (JMC) for visualization and analysis.
- Available for free in OpenJDK 11+ (was commercial in Oracle JDK 8).

SUMMARY	*Java Flight Recorder = built-in, low-overhead JVM profiler. Records runtime events for post-analysis. Use with JDK Mission Control.*

★ FAQ Q56. Discuss Java Generics.

- Generics enable type-safe parameterized classes, interfaces, and methods.
- Compile-time type checking eliminates runtime ClassCastException.
- Type Erasure: generic type information is erased at compile time; replaced with Object or bounds.
- Wildcards: <?> unbounded, <? extends T> upper-bounded (read), <? super T> lower-bounded (write).
- PECS principle: Producer Extends, Consumer Super.

SUMMARY	*Generics = compile-time type safety. Type erasure removes info at runtime. Wildcards enable flexible APIs. PECS guides wildcard choice.*

★ FAQ Q57. Young vs Old Generation memory?

Area	Description
Young Generation	Where new objects are allocated. Divided into Eden and two Survivor spaces. Minor GC is frequent and fast.
Old (Tenured) Generation	Long-lived objects promoted from Young gen. Major/Full GC is less frequent but slower.
Metaspace	Class metadata (replaced PermGen in Java 8). Grows automatically from native memory.
Minor GC	Collects Young gen only. Quick. Survivor spaces used for promotion.
Full GC	Collects entire heap. Slower, causes STW (Stop-The-World) pause.
SUMMARY	*Young Gen: new short-lived objects + Minor GC. Old Gen: long-lived + Major GC. Metaspace: class metadata (post Java 8).*

Serialization

Q58. How to prevent fields from serialization?

- Mark field as transient — excluded from serialization output.
- Alternatively, customize via writeObject()/readObject() and skip the field.
- static fields are never serialized (class state, not object state).

SUMMARY	*Use transient keyword to exclude fields from serialization. Static fields are inherently excluded.*

Q59. Can class be serialized if member is not serializable?

- No — if a non-serializable instance field exists and is not transient, serialization throws NotSerializableException.
- Solution: mark the problematic field as transient.
- Exception: static fields are fine (not serialized); null non-serializable references are technically OK.

SUMMARY	*Non-serializable member → NotSerializableException. Fix: mark it transient. Static fields are unaffected.*

Q60. Difference between writeObject() and readObject()?

- writeObject(ObjectOutputStream): custom serialization logic — write fields manually.
- readObject(ObjectInputStream): custom deserialization — read fields back in same order.
- Used when default serialization is insufficient (e.g., field encryption, versioning).
- Must be declared private and match in structure for correct round-trip.

SUMMAR	*writeObject/readObject: custom serialization hooks. Write fields manually; read in same order. Used for encryption, versioning.*

Y

Q61. Can static fields be serialized?

- No. Static fields belong to the class, not the instance; they are never part of the serialized stream.
- Upon deserialization, static fields retain their current JVM values, not restored from stream.

SUMMARY

Static fields are NOT serialized — they are class-level state, not object state. Retained from current JVM on deserialization.

Q62. How to handle circular references?

- Java's ObjectOutputStream handles circular references automatically via reference tracking.
- It maintains a table of already-serialized objects; back-references are written as handles (not recursively).
- Custom serialization with writeObject/readObject can also manage them explicitly.

SUMMARY

Java automatically handles circular references during serialization via object reference table. No manual handling needed for standard cases.

Overloading / Overriding

★ FAQ Q63. What is method overloading?

- Same method name, different parameter signature (type, count, or order) in the same class.
- Resolved at compile time — static/early binding.
- Return type alone is NOT sufficient for overloading.

SUMMARY

Overloading = same name, different parameters in same class. Compile-time resolution. Improves readability with consistent naming.

Q64. How compiler selects overloaded method?

- Phase 1: Exact match.
- Phase 2: Widening (int fits into long, float into double).
- Phase 3: Autoboxing/unboxing.
- Phase 4: Variable arguments (varargs) — lowest priority.
- Ambiguity: if two methods are equally applicable, compile error.

SUMMAR

Compiler resolves overloads by: exact match → widening → autoboxing → varargs. Ambiguous calls = compile error.

Y

Q65. Can overloading differ only by return type?

- No. Return type alone is insufficient for overloading.
- Methods must differ in parameter type, count, or order.
- If two methods differ only in return type, the compiler cannot distinguish them — compile error.

SUMMARY

Cannot overload by return type alone. Parameters must differ. Same name + same params + different return = compile error.

Q66. Rules for method overloading?

- Same method name in the same class.
- Different parameter lists (type, number, or order).
- Return type can be same or different (but alone is not enough).
- Access modifiers can be same or different.
- Can throw same or different exceptions.

SUMMARY

Overloading rules: same name, different parameters (type/count/order), in same class. Return type, access, exceptions are irrelevant.

★ FAQ Q67. What is method overriding?

- Subclass provides a new implementation for a method already defined in its superclass.
- Method signature (name + parameters) must match exactly.
- Runtime polymorphism via dynamic dispatch.

SUMMARY

Overriding: subclass redefines superclass method with same signature. Enables runtime polymorphism.

Q68. Rules for method overriding?

- Same method name, parameter list, and return type (or covariant subtype).
- Cannot reduce access visibility (public can't become protected/private).
- Cannot override final, static, or private methods.
- Overriding method can only throw narrower or fewer checked exceptions.
- @Override annotation strongly recommended.

SUMM

Override rules: same signature, not stricter access, covariant return, fewer/narrower checked exceptions. Cannot override final/static/private.

A
R
Y

Step 3 – Core Java — Level III (Advanced)

Core / Design

Q1. Scenario using PriorityQueue?

- PriorityQueue is a heap-based queue where elements are ordered by natural order or a custom Comparator.
- The head of the queue is always the smallest (min-heap by default).
- Use cases: task scheduling by priority, Dijkstra's algorithm, event-driven simulation, top-K problems.

```
PriorityQueue<Integer> pq = new PriorityQueue<>();
pq.add(5); pq.add(1); pq.add(3); System.out.println(pq.poll()); // 1
```

SUMMARY	*PriorityQueue = min-heap. poll() always returns smallest element. Use for task scheduling, algorithms (Dijkstra), top-K problems.*

Q2. What are enums and their use?

- Enums define a fixed set of named constants as a type.
- Can have fields, methods, abstract methods, and constructors.
- values() returns all constants; ordinal() returns position; name() returns string name.
- EnumSet and EnumMap are highly efficient collections for enums.
- Enums are inherently thread-safe singletons.

SUMMARY	*Enum = type-safe named constants. Can contain fields/methods. Thread-safe. Use EnumSet/EnumMap for performance.*

★ FAQ Q3. Builder Pattern vs Factory Pattern?

Pattern	Description
Factory	Creates objects without exposing creation logic. Returns instances via factory method. Best for: creating objects of varying types from a common interface.
Builder	Constructs complex objects step by step. Separates construction from representation. Best for: objects with many optional parameters.
Example	Factory: Shape.create("circle"). Builder: new Pizza.Builder().size(12).toppings(CHEESE).build()
Java Example	Factory: Calendar.getInstance(). Builder: StringBuilder, Stream.Builder, Locale.Builder
SUMMARY	*Factory: object creation without specifying exact class. Builder: step-by-step construction of complex objects with optional params.*

Q4. Impact of final method on inheritance?

- A final method in a superclass cannot be overridden by any subclass.
- Inheritance still works — subclass inherits the method but cannot override it.
- Useful for: securing critical logic, enabling JVM inlining optimizations.

SUMMARY	*final method: inherited but cannot be overridden. Inheritance still occurs. Secures behavior and allows JVM inlining.*

Q5. Can overloading be determined at runtime?

- No. Method overloading is resolved at compile time (static binding).
- The compiler selects the best matching overloaded method based on the declared (reference) type, not actual type.
- Only method overriding is determined at runtime (dynamic dispatch based on object type).

SUMMARY	*Overloading: compile-time (based on reference type). Overriding: runtime (based on actual object type). They are distinct mechanisms.*

Q6. How Java resolves overloaded methods?

- Compiler picks the most specific method in three phases: exact match → widening → boxing/unboxing → varargs.
- Uses the declared (static) type of arguments, not the runtime type.
- Ambiguity between equally-specific overloads results in compile error.

SUMMARY	*Java resolves overloads at compile time using static type. Most specific match wins: exact → widening → boxing → varargs.*

Q7. What is diamond operator?

- Introduced in Java 7. The diamond operator <> enables type inference for generic instantiation.
- Eliminates redundant type specification on the right-hand side.

```
List<Map<String, List<Integer>>> old = new ArrayList<Map<String, List<Integer>>>();
List<Map<String, List<Integer>>> modern = new ArrayList<>(); // diamond
```

SUMMARY	*Diamond operator <> infers generic types from context. Available since Java 7. Reduces verbose generic type repetition.*

★ FAQ Q8. Explain inner classes.

- Non-static inner class: has access to outer class instance members; needs outer instance to be created.
- Static nested class: does not need outer instance; can only access outer class static members.
- Local inner class: defined inside a method; scope limited to that method.
- Anonymous inner class: one-time use, no name, defined inline. Used before lambdas.

SUMMARY

Inner classes: non-static (access outer instance), static nested (no outer needed), local (method scope), anonymous (one-time use).

Q9. Can inner classes have static members?

- Non-static inner classes cannot have static members (except static final constants).
- Static nested classes can have static members (they behave like top-level classes in that regard).
- Local and anonymous inner classes cannot have static members.

SUMMARY

Non-static inner classes cannot have static members (except static final). Static nested classes can. Rule: static members need static context.

Q10. What is anonymous inner class?

- An anonymous inner class is a class without a name, defined and instantiated in a single expression.
- Extends a class or implements an interface on the fly.
- Common before Java 8 for event listeners and callbacks.
- In Java 8+, largely replaced by lambda expressions for functional interfaces.

```
Runnable r = new Runnable() { public void run() { System.out.println("Running"); } };
```

SUMMARY

Anonymous inner class: nameless, single-use class. Largely replaced by lambdas in Java 8+. Still needed for multi-method interfaces.

Advanced OOP / Packages

Q11. How Java determines overloaded method?

- At compile time, uses static (declared) type of argument, not runtime type.
- Selects most specific applicable method through: exact match → widening → boxing → varargs.

SUMMARY

Overloaded method selection: compile-time, based on declared argument types. Most specific applicable method is chosen.

Q12. What if two packages have same class name?

- Both can coexist. Use fully qualified class name to distinguish: java.util.Date vs java.sql.Date.
- When importing both, use fully qualified name for the ambiguous one.

```
import java.util.Date;
java.sql.Date sqlDate = new java.sql.Date(System.currentTimeMillis());
```

SUMMARY

Same class name in different packages: use fully qualified names to disambiguate. Import one; use FQN for the other.

Q13. How to access package-private class from another package?

- Default (package-private) access means only classes in the same package can access it.
- Cannot be accessed from another package without changing access modifier to public.
- In Java 9+: module system's exports clause controls package-level access between modules.

SUMMARY

Package-private access: not accessible from other packages. Change to public or use Java 9 modules to expose specific packages.

Q14. Can we modify final object reference?

- final prevents reassigning the reference variable itself, but the object it points to can still be mutated.

```
final List<String> list = new ArrayList<>();
list.add("Java"); // OK — object is mutable
list = new ArrayList<>(); // COMPILE ERROR — reference is final
```

SUMMARY

final reference cannot be reassigned, but the object itself is still mutable. final ≠ immutable object.

Q15. Default access modifier?

- Default (no keyword) = package-private access.
- Accessible only within the same package.
- More restrictive than protected (which allows subclass access from other packages).

SUMMARY

Default/package-private: accessible only within same package. More restrictive than protected but less than private.

★ FAQ Q16. Issues with mutable keys in HashMap?

- HashMap stores entries based on key's hashCode() and equals().
- If a key's state changes after insertion, its hashCode changes.
- The entry is now in the wrong bucket — get() will not find it; lost entry.
- Best practice: use immutable keys (String, Integer, or custom immutable class).

SUMMARY

Mutable HashMap keys cause entries to be "lost" when hashCode changes after insertion. Always use immutable keys.

Concurrency

★ FAQ Q17. Explain Java concurrency model.

- Based on shared memory with explicit synchronization.
- Java Memory Model (JMM) defines happens-before rules for thread visibility.
- Tools: synchronized, volatile, java.util.concurrent (locks, executors, atomics, collections).
- Thread states: NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING, TERMINATED.
- Modern: CompletableFuture, Reactive Streams, Virtual Threads (Java 21).

SUMMARY

Java concurrency: shared memory + explicit synchronization. JMM governs visibility. java.util.concurrent provides high-level tools.

Q18. Challenges in thread management?

- Race conditions: multiple threads modifying shared state concurrently.
- Deadlock: two threads each holding a lock and waiting for the other.
- Livelock: threads keep responding to each other without making progress.
- Starvation: low-priority threads never get CPU time.
- Visibility issues: CPU caching causing threads to see stale data.
- Context-switching overhead with many threads.

SUMMARY

Challenges: race conditions, deadlock, livelock, starvation, visibility issues. Use high-level concurrency utilities to mitigate.

★ FAQ Q19. Can volatile replace synchronization?

- Partially. volatile ensures visibility (reads/writes go to main memory).
- volatile does NOT ensure atomicity of compound operations (check-then-act, i++).
- synchronized ensures both visibility AND atomicity.
- Use volatile for: simple flags, status variables where only visibility is needed.

- Use synchronized or AtomicInteger for compound operations.

SUMMARY	*volatile: visibility only. synchronized: visibility + atomicity. Cannot replace synchronized for compound operations.*

Q20. Can deadlock occur with single thread?

- No. Deadlock requires at least two threads each waiting for a lock held by the other.
- A single non-reentrant lock can cause self-deadlock if one thread tries to acquire a lock it already holds, but Java's intrinsic locks are reentrant.
- ReentrantLock is also reentrant by design.

SUMMARY	*Deadlock requires 2+ threads in circular wait. Single thread cannot deadlock (Java locks are reentrant). Self-deadlock impossible in Java.*

★ FAQ Q21. Synchronized vs concurrent collections?

Aspect	Synchronized vs Concurrent Collections
Synchronized	Collections.synchronizedList/Map. Entire collection locked per operation. Poor concurrency.
ConcurrentHashMap	Segment/stripe locking (Java 7) or CAS + synchronized on individual buckets (Java 8+). High concurrency.
CopyOnWriteArrayList	New copy of array on every write. Reads lock-free. Best for read-heavy, rare-write lists.
BlockingQueue	Thread-safe; supports blocking put/take for producer-consumer.
SUMMARY	*Prefer java.util.concurrent collections over synchronized wrappers. ConcurrentHashMap/CopyOnWriteArrayList offer far better throughput.*

Q22. How Java handles multithreading?

- Thread creation: Thread class, Runnable, Callable, ExecutorService, ForkJoinPool.
- Synchronization: synchronized, Lock, ReadWriteLock, StampedLock.
- Atomics: AtomicInteger, AtomicReference (lock-free CAS operations).
- High-level: CompletableFuture for async pipelines; Virtual Threads (Java 21) for massive concurrency.

SUMMARY	*Java multithreading: Thread/Runnable, ExecutorService for pooling, synchronized/Lock for safety, Atomic classes for lock-free ops.*

Executor Framework

★ FAQ Q23. Role of ExecutorService?

- ExecutorService manages a pool of worker threads, decoupling task submission from execution.
- Key implementations: ThreadPoolExecutor (customizable), Executors.newFixedThreadPool(), newCachedThreadPool(), newSingleThreadExecutor(), newScheduledThreadPool().
- submit(): accepts Callable/Runnable, returns Future for async result retrieval.
- shutdown(): stops accepting new tasks; shutdownNow(): attempts to stop running tasks.

SUMMARY	*ExecutorService: manages thread pool. Decouple task submission from execution. Use shutdown() when done to release resources.*

Q24. Difference between submit() and execute()?

Method	submit() vs execute()
execute(Runnable)	Fire-and-forget. No return value. Uncaught exceptions bubble to UncaughtExceptionHandler.
submit(Callable/Runnable)	Returns Future<T>. Exceptions wrapped and thrown when Future.get() is called.
Use case	execute: when you don't need result. submit: when you need result or to handle exceptions properly.
SUMMARY	*execute(): void, no future, exceptions uncaught. submit(): returns Future, exceptions captured in Future.get(). Use submit for most cases.*

Q25. What is RejectedExecutionHandler?

- Called when ExecutorService cannot accept a new task (pool saturated + queue full, or executor shut down).
- Built-in policies: AbortPolicy (default, throws RejectedExecutionException), CallerRunsPolicy (caller thread executes task), DiscardPolicy (silently discards), DiscardOldestPolicy (discards oldest queued task).

SUMMARY	*RejectedExecutionHandler handles overflow when thread pool is saturated. Default=AbortPolicy (throws). CallerRunsPolicy is gentler alternative.*

Collections / Concurrency

★ FAQ Q26. How ConcurrentHashMap works internally?

- Java 7: Divided into 16 segments (Segment extends ReentrantLock). Each segment independently locked.
- Java 8+: Uses Node[] array + synchronized on individual buckets + CAS operations.
- Reads are lock-free (volatile Node reads). Writes lock only the specific bucket.
- Treeification: buckets with 8+ entries convert to red-black tree (TreeNode) for O(log n) lookup.
- Does NOT allow null keys or null values (unlike HashMap).

S	*CHM Java 8+: lock-free reads (volatile), per-bucket synchronization for writes. No nulls. Treeifies at 8 entries.*

SUMMARY	*High concurrency.*

★ FAQ Q27. synchronized vs ReentrantLock?

Feature	synchronized vs ReentrantLock
Try lock	synchronized: no. ReentrantLock: tryLock(), tryLock(timeout).
Interruptible	synchronized: no. ReentrantLock: lockInterruptibly().
Fairness	synchronized: no guarantee. ReentrantLock: optional fair mode (FIFO).
Conditions	synchronized: one condition (wait/notify). ReentrantLock: multiple Conditions.
Explicit unlock	synchronized: auto-release. ReentrantLock: must unlock() in finally.
SUMMARY	*ReentrantLock: more flexible (tryLock, timeout, interruptible, multiple conditions, fairness). synchronized: simpler, auto-release. Prefer ReentrantLock for advanced scenarios.*

Q28. What happens if exception occurs in synchronized block?

- When an exception propagates out of a synchronized block/method, the lock is automatically released.
- No risk of lock being permanently held due to an exception.
- With ReentrantLock, must explicitly call unlock() in finally to ensure release.

SUMMARY	*Exception in synchronized block: lock is automatically released. With ReentrantLock, always unlock in finally to prevent lock leaks.*

Java 8 Streams

★ FAQ Q29. How to create infinite stream?

- Stream.iterate(seed, f): produces infinite stream by repeatedly applying function.
- Stream.generate(Supplier): produces infinite stream from a supplier (e.g., random numbers).
- Always use with limit() or findFirst() to terminate; otherwise runs forever.

```
Stream.iterate(0, n -> n+2).limit(10).forEach(System.out::println); // 0,2,4,...18
Stream.generate(Math::random).limit(5).forEach(System.out::println);
```

SUMMARY	*Infinite streams: Stream.iterate() (sequential progression) or Stream.generate() (supplier). Always limit with limit()/findFirst()/takeWhile().*

Q30. What is Function interface?

- java.util.function.Function<T,R>: takes one argument of type T, returns result of type R.

- Method: apply(T t).
- Composition: andThen(Function after), compose(Function before).

```
Function<String, Integer> len = String::length;
Function<Integer, Integer> doubleIt = n -> n * 2;
len.andThen(doubleIt).apply("hello"); // 10
```

SUMMARY *Function<T,R>: T → R. Use apply(). Chainable via andThen()/compose(). Core functional interface for transformation pipelines.*

★ FAQ Q31. What are method references?

- Method references (::) are compact lambdas that refer to existing methods.
- Types: Static (ClassName::staticMethod), Instance (instance::method), Unbound (ClassName::instanceMethod), Constructor (ClassName::new).

```
list.forEach(System.out::println);          // instance method reference
list.stream().map(String::toUpperCase);     // unbound instance method reference
stream.map(Person::new);                    // constructor reference
```

SUMMARY *Method references (::) are shorthand lambdas. 4 types: static, instance, unbound, constructor. More readable than equivalent lambdas.*

Q32. How to sort using streams?

- Use sorted() for natural ordering; sorted(Comparator) for custom order.

```
list.stream().sorted().collect(Collectors.toList());
list.stream().sorted(Comparator.comparing(Person::getAge).reversed()).collect(Collect
ors.toList());
```

- Multi-level sort: Comparator.comparing(...).thenComparing(...)

SUMMARY *Stream.sorted(): natural order. sorted(Comparator): custom. Chain with thenComparing() for multi-level sorts.*

Q33. Use of reduce()?

- reduce() combines elements of a stream into a single result using a BinaryOperator.
- Forms: reduce(identity, accumulator) — returns T; reduce(accumulator) — returns Optional<T>.

```
int sum = IntStream.rangeClosed(1,10).reduce(0, Integer::sum); // 55
Optional<Integer> product = list.stream().reduce((a,b) -> a*b);
```

SUMMAR *reduce(): aggregates stream to single value. With identity: always returns value. Without identity: returns Optional (handles empty stream).*

Y

Q34. How filter() works?

- filter(Predicate<T>) is a stateless intermediate operation that passes only elements matching the predicate.
- Returns a new Stream; original is unchanged.
- Lazy: elements are only processed when a terminal operation is called.

```
list.stream().filter(s -> s.startsWith("J")).collect(Collectors.toList());
```

SUMMARY *filter(Predicate): lazy intermediate op passing only matching elements. Returns new stream. Applied when terminal op triggers.*

Q35. Use of Collectors.toList()?

- Collectors.toList() is a terminal collector that accumulates stream elements into a List.
- Java 16+: Stream.toList() provides an unmodifiable list directly.
- For specific list type: Collectors.toCollection(ArrayList::new).

```
List<String> result = stream.filter(...).map(...).collect(Collectors.toList());
```

SUMMARY *Collectors.toList(): accumulates stream into a mutable List. Java 16+: .toList() returns unmodifiable List. Use toCollection() for specific type.*

Serialization / Generics

Q36. Can enum extend class?

- No. All enums implicitly extend java.lang.Enum, which is already their superclass.
- Java does not support multiple inheritance for classes.
- Enums CAN implement interfaces.

SUMMARY *Enums cannot extend classes (already extend java.lang.Enum). They CAN implement interfaces.*

Q37. How to iterate enum values?

- EnumType.values() returns an array of all enum constants.

```
for (Day d : Day.values()) { System.out.println(d); }
```

- EnumSet.allOf(Day.class) — efficient set-based iteration.
- Can also use EnumSet.range(Day.MON, Day.FRI).

SU *Iterate enums: values() for all, EnumSet.allOf() for set operations. values() returns array; EnumSet for efficient range/subset iteration.*

MMARY	

Q38. Can static fields be serialized?

- No. Static fields are class-level state, not part of the object graph; they are excluded from serialization.
- On deserialization, static fields retain whatever value they currently have in the JVM.

SUMMARY	*Static fields not serialized — class state, not instance state. Retain JVM-current values on deserialization.*

Q39. What happens if exception during serialization?

- If an exception occurs during serialization, the stream may be in a partially written state.
- The stream cannot be used for further serialization.
- InvalidClassException if class structure has changed incompatibly (serialVersionUID mismatch).
- NotSerializableException if a non-serializable, non-transient field is encountered.

SUMMARY	*Exception during serialization: stream is corrupted, can't continue. NotSerializableException for non-serializable fields; InvalidClassException for version mismatch.*

Q40. Serializable class with non-serializable member?

- Throws NotSerializableException at runtime when trying to serialize.
- Solution 1: Mark the field transient.
- Solution 2: Make the member class implement Serializable.
- Solution 3: Custom writeObject/readObject to handle the field manually.

SUMMARY	*Non-serializable field → NotSerializableException. Fix: make it transient, make it Serializable, or use custom writeObject/readObject.*

★ FAQ Q41. What is Type Erasure?

- Type Erasure is the process by which the Java compiler removes generic type information from bytecode.
- Generic types become Object (or the upper bound) in bytecode.
- Purpose: backward compatibility with pre-generics (Java 1.4) code.
- Consequence: Cannot use instanceof with generic types at runtime; cannot create generic arrays.
- Bridge methods may be generated by compiler to preserve polymorphism.

S	*Type Erasure: generic type info removed at compile time for bytecode compatibility. Cannot check generic*

SUMMARY

types at runtime. Bridge methods fill the gap.

★ FAQ Q42. What is generic type inference?

- The compiler deduces type arguments from context to avoid explicit specification.
- Method type inference: compiler infers T from argument types.
- Diamond operator <>: infers generic type from left-hand side declaration.
- Target type inference: infers type from assignment context.

```
List<String> list = new ArrayList<>();          // diamond inference
<T> T identity(T t) { return t; }
String s = identity("hello");                   // T inferred as String
```

SUMMARY

Type inference: compiler deduces generic types from context. Diamond operator, method arguments, and target types all provide inference clues.

★ QUICK REFERENCE CHEAT SHEET ★

JDK / JVM / JRE	
JDK	Full dev toolkit: compiler (javac) + JRE + debugging tools.
JRE	Runtime: JVM + core libraries. Runs programs, no compilation.
JVM	Executes bytecode. Manages memory, GC, JIT compilation.
JIT	Just-In-Time compiler converts hot bytecode to native code.
Class Loading	Bootstrap → Extension → Application ClassLoader (delegation model).

OOP Pillars	
Encapsulation	Private fields + public getters/setters. Data hiding.
Inheritance	extends (class), implements (interface). Single class inheritance.
Polymorphism	Overloading (compile-time) + Overriding (runtime dispatch).
Abstraction	abstract class (partial impl) + interface (pure contract).

Keywords Quick Reference	
static	Class-level; no instance needed. Shared variable, utility method, static block.
final	Variable: constant. Method: no override. Class: no subclass.
volatile	Memory visibility across threads. Not atomic!
transient	Exclude field from serialization.
synchronized	Mutual exclusion. Locks object monitor.
abstract	Incomplete; must be overridden. Cannot be instantiated.
this	Current object reference. Constructor chaining via this().
super	Parent class reference. Parent constructor via super().

== vs .equals() vs hashCode()	
==	Reference equality for objects; value equality for primitives.
.equals()	Logical/content equality. Override in custom classes.
hashCode()	Must be consistent with equals(). Equal objects must have same hashCode.
Contract	If a.equals(b) → a.hashCode() == b.hashCode() (but not vice versa).

Collections Cheat Sheet	
ArrayList	Dynamic array. O(1) access, O(n) insert/delete. Not thread-safe.
LinkedList	Doubly-linked. O(1) insert/delete, O(n) access. Also implements Deque.
HashSet	Hash-based Set. O(1) ops. No order. Allows one null.
TreeSet	Red-black tree. O(log n). Sorted order. No null.
HashMap	Hash table. O(1) avg. Allows null key/value. Not thread-safe.
TreeMap	Red-black tree Map. Sorted by key. O(log n). No null key.
ConcurrentHashMap	Thread-safe HashMap. Lock-free reads, bucket-level write locks. No nulls.
PriorityQueue	Min-heap. O(log n) insert/poll. O(1) peek.

Java 8 Features	
Lambdas	(params) -> body. Concise anonymous functions.
Streams	filter(), map(), reduce(), collect(). Lazy, chainable pipeline.
Optional<T>	Container: Optional.of(), empty(), map(), orElse(), ifPresent().
Default Methods	Interface default method { ... }. Backward-compatible API evolution.

Method Refs	ClassName::method. Static, instance, unbound, constructor.
Date/Time API	LocalDate, LocalTime, LocalDateTime, ZonedDateTime, Duration, Period.
CompletableFuture	thenApply(), thenCompose(), allOf(), anyOf() for async chaining.

Concurrency Quick Ref	
Thread states	NEW → RUNNABLE → BLOCKED/WAITING/TIMED_WAITING → TERMINATED
synchronized	Intrinsic lock. Reentrant. Auto-released on exception.
volatile	Visibility guarantee. Not for compound ops (i++).
ReentrantLock	tryLock(), lockInterruptibly(), multiple Conditions, fair mode.
AtomicInteger	CAS-based lock-free int ops. compareAndSet(), incrementAndGet().
ExecutorService	Thread pool. submit() → Future; execute() → void. shutdown() after use.
CompletableFuture	Async pipeline. thenApply(sync), thenApplyAsync(async pool).
Virtual Threads	Java 21 Project Loom. Millions of lightweight threads.

Exception Hierarchy	
Throwable	Root. Error + Exception.
Error	JVM-level: OutOfMemoryError, StackOverflowError. Do not catch.
Checked	Must handle/declare: IOException, SQLException, ClassNotFoundException.
Unchecked	RuntimeException subclasses: NPE, AIOOBE, ClassCastException.
try-with-resources	Auto-closes AutoCloseable. Java 7+. Preferred over manual close.

Java Version Highlights	
Java 8 (LTS)	Lambdas, Streams, Optional, default methods, new Date/Time API.
Java 11 (LTS)	String methods (strip, isBlank), HttpClient, var in lambdas, local-var syntax.
Java 17 (LTS)	Sealed classes, records, pattern matching instanceof, text blocks.
Java 21 (LTS)	Virtual Threads, sequenced collections, pattern matching for switch, record patterns.

#1
JDK vs JRE vs JVM differences
#2
== vs .equals() vs hashCode() contract
#3
HashMap internal working (hash, bucket, treeify)
#4
Java 8 features (Lambda, Streams, Optional)
#5
Checked vs Unchecked exceptions
#6
Abstract class vs Interface
#7
final vs volatile vs synchronized
#8
Overloading vs Overriding
#9
ConcurrentHashMap internals (vs HashMap vs Hashtable)
#10
Garbage Collection (G1GC, Young/Old gen)
#11
Stream intermediate vs terminal operations

#12
Serialization, transient, serialVersionUID
#13
Builder / Factory / Singleton design patterns
#14
Virtual Threads (Java 21 Project Loom)
#15
Type Erasure and Generics